AA/EE/ME 548

Linear Multivariable Control

State-space representation

Spring 2026

State-space representation

- $x \in \mathbb{R}^n$: State vector
- $u \in \mathbb{R}^m$: Control input vector
- $d \in \mathbb{R}^d$: Disturbance input vector

Continuous-time notation

- $x(t)$ means the state at time t. Similarly for $u(t)$ and $d(t)$

Discrete-time notation (timestep size Δt)

- t_k means time at step k
- $x_k = x(t_k)$ means the state at time t_k. Similarly for u_k and d_k

Note: Will drop the disturbance term. Will show up later in robust control discussion.

Dynamics

Continuous-time (ODE)

$$\dot{x} = f_c(x, u, t)$$

Discrete-time (Difference)

$$x_{k+1} = f_d(x_k, u_k, t_k)$$

Types of dynamics (also applies for discrete-time)

- Nonlinear: $f(x, u, t)$ consists of nonlinear elements
- Control affine: $f_0(x, t) + B(x, t)u$
- Linear: $A(t)x + B(t)u$
- Time-invariant: Drop the t dependency

From continuous-time to discrete-time

Suppose we start with continuous-time dynamics (derived from EOM). That is, we know $\dot{x} = f_c(x, u, t)$.

We may desire to obtain discrete-time dynamics (useful for mathematical optimization). Expressed in several ways

$$x_{k+1} = x_k + \int_{t_k}^{t_{k+1}} f_c(x(\tau), u(\tau), \tau) d\tau$$

$$x(t_k + \Delta t) = x(t_k) + \int_{t_k}^{t_k + \Delta t} f_c(x(\tau), u(\tau), \tau) d\tau$$

$$x_{k+1} = x_k + \int_{0}^{\Delta t} f_c(x(t_k + \tau), u(t_k + \tau), t_k + \tau) d\tau$$

Integrating dynamics

- Performing integration analytically is tedious and slow
  - Symbolic packages, by hand 😱
- Numerical integration is preferred (fast)
  - Euler, Runge-Kutta
- Need to know $u(\cdot)$

$$x_{k+1} = x_k + \int_{t_k}^{t_{k+1}} f_c(x(\tau), u(\tau), \tau) d\tau$$

Zero-order hold

- Assume $u(\cdot)$ is constant over Δt

- First-order hold? Second-order hold?

Linearization

We would like to work with *linear* dynamics (or affine). Why?

- Linear dynamics are easy and tractable to work with (547)
- Affine dynamics mean affine constraints which are convex

Apply first-order Taylor approximation on the dynamics (okay both continuous and discrete-time)

Linearization (Taylor series)

Consider function near an operating points x_0

Scalar-valued function: $f: \mathbb{R} \to \mathbb{R}$

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2!} f''(x_0)(x - x_0)^2 + \cdots + \frac{1}{k!} f^{(k)}(x - x_0)^k + \cdots$$

Scalar-valued function with vector input: $f: \mathbb{R}^n \to \mathbb{R}$

$$f(x) \approx f(x_0) + \nabla f(x_0)^T (x - x_0) + \frac{1}{2!} (x - x_0)^T H_f(x_0)(x - x_0) + \cdots$$

Linearization

Vector-valued function with vector input: $f: \mathbb{R}^n \to \mathbb{R}^m$

$$f(x) \approx f(x_0) + J_f(x_0)(x - x_0) + \cdots$$

What about function with two arguments?

What about $f: \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$?

$$f\left(\begin{bmatrix} x \\ u \end{bmatrix}\right) \approx f\left(\begin{bmatrix} x_0 \\ u_0 \end{bmatrix}\right) + J_f\left(\begin{bmatrix} x_0 \\ u_0 \end{bmatrix}\right)\left(\begin{bmatrix} x \\ u \end{bmatrix} - \begin{bmatrix} x_0 \\ u_0 \end{bmatrix}\right) + \cdots$$

$$f\left(\begin{bmatrix} x \\ u \end{bmatrix}\right) \approx f\left(\begin{bmatrix} x_0 \\ u_0 \end{bmatrix}\right) + \qquad \left(\begin{bmatrix} x \\ u \end{bmatrix} - \begin{bmatrix} x_0 \\ u_0 \end{bmatrix}\right) + \cdots$$

$$f(x,u) \approx f(x_0,u_0) + \nabla_x f(x_0,u_0)^T (x - x_0) + \nabla_u (x_0,u_0)^T (u - u_0) \ldots$$

Linearizing nonlinear dynamics

$$f(x,u) \approx f(x_0,u_0) + \nabla_x f(x_0,u_0)^T (x - x_0) + \nabla_u (x_0,u_0)^T (u - u_0)$$

Rearranging…

$$f(x,u) \approx \nabla_x f(x_0,u_0)^T x + \nabla_u (x_0,u_0)^T u + f(x_0,u_0) - \nabla_x f(x_0,u_0)^T x_0 - \nabla_u (x_0,u_0)^T u_0$$

What if $f(x_0,u_0) = 0$ and $\delta x = x - x_0, \delta u = u - u_0$?

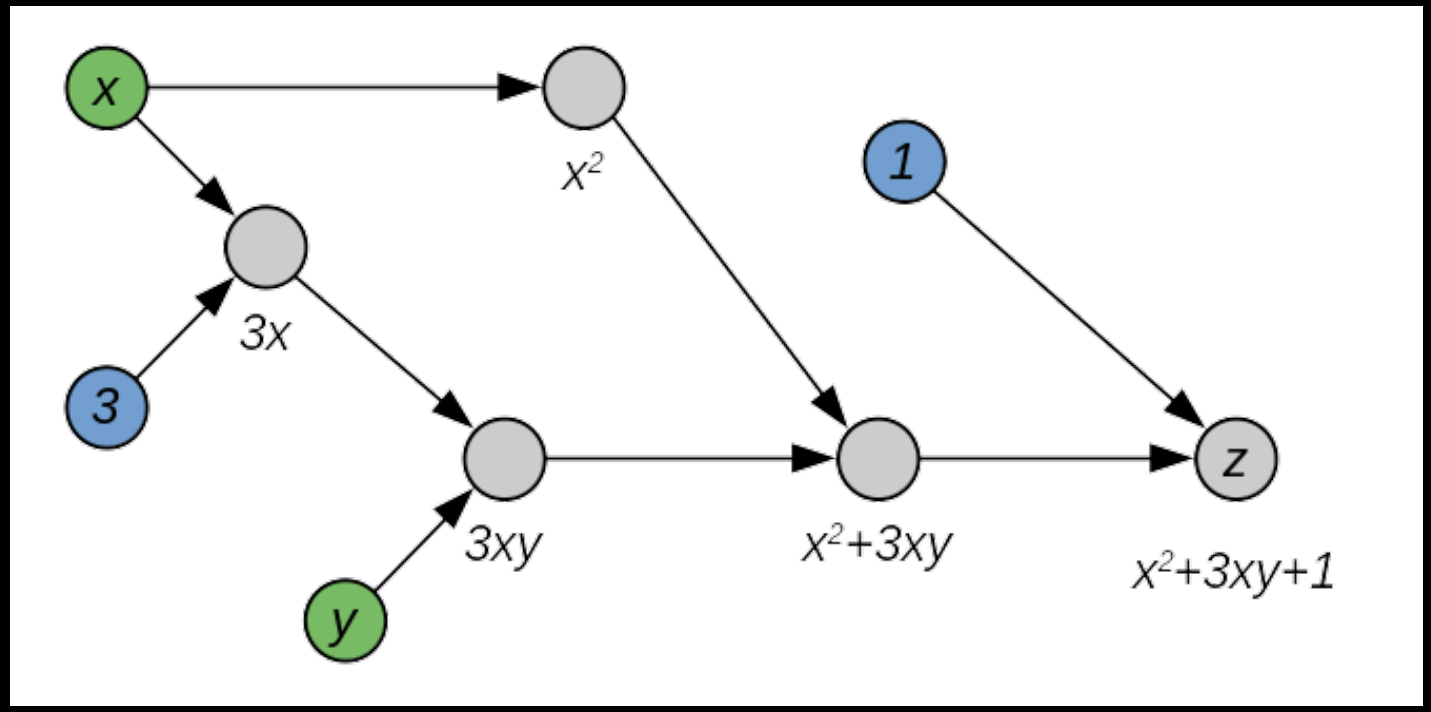
What about linearizing discrete-time dynamics obtained by integrating continuous-time dynamics?

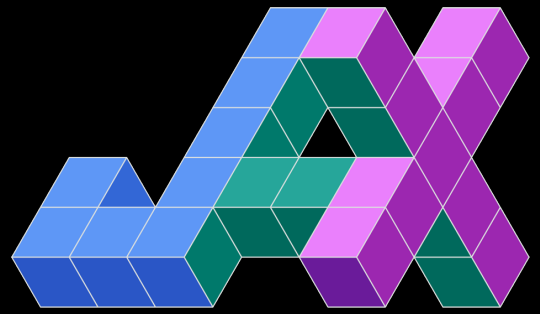
Need $\nabla_x f(x_0, u_0)$ and $\nabla_u f(x_0, u_0)$. How to get?

Finite differencing?

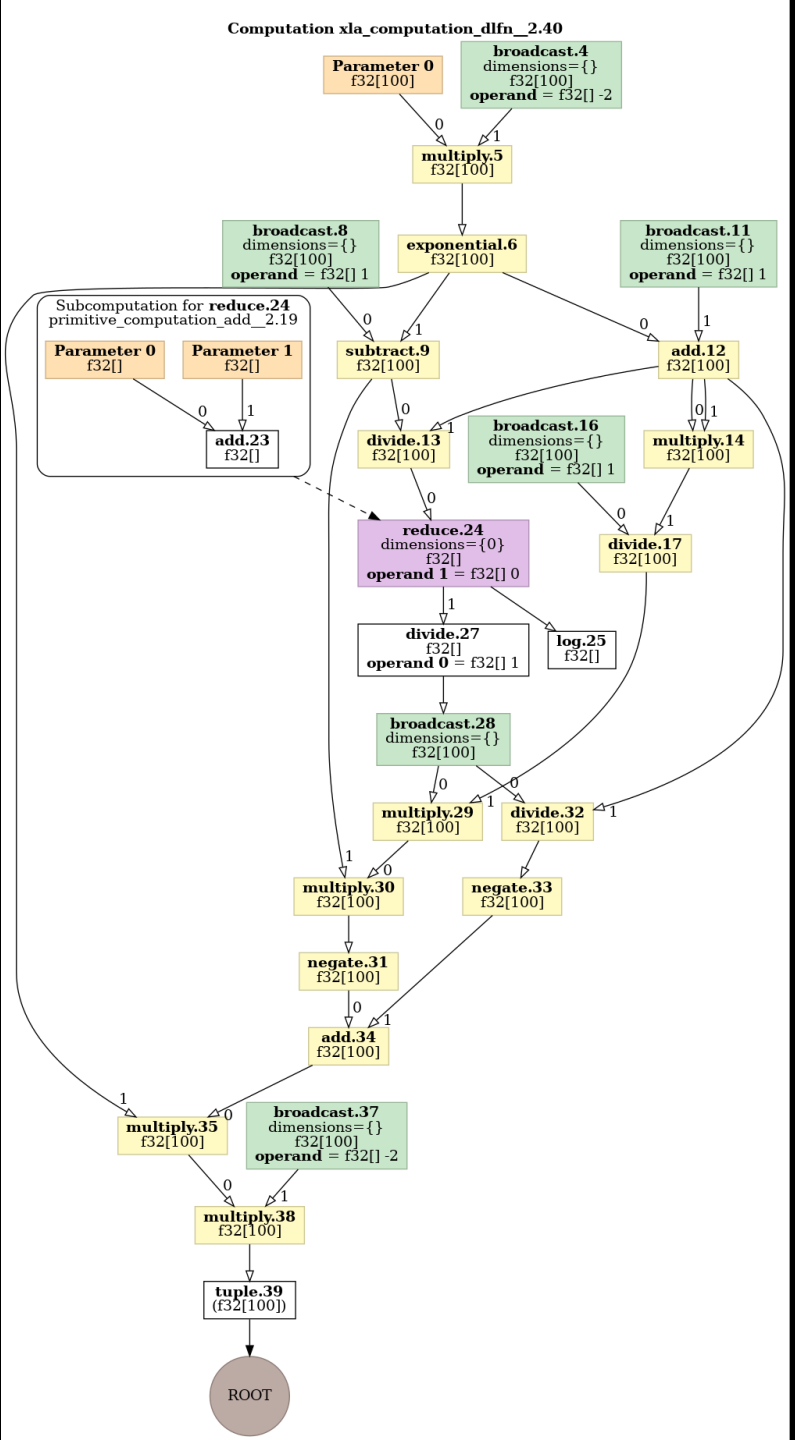
Analytically?

Automatic differentiation





JAX is a library for array-oriented numerical computation (*à la* NumPy), with automatic differentiation and JIT compilation to enable high-performance machine learning research.



https://docs.jax.dev/en/latest/notebooks/thinking_in_jax.html

Functional vs imperative programming

Is your code a *description of a mathematical mapping* (functional), or a *list of instructions for a machine* (imperative)?

```python
import jax.numpy as jnp
def square(x):
    return x**2

A = jnp.array([1, 2, 3])
B = jax.vmap(square)(A) # A is untouched; B is a new array
```

Functional programming

```
for i = 1:length(A)
    A(i) = A(i)^2; % Modifying the state of A in place
end
```

Imperative programming

See Homework 1 for getting up to speed with JAX